Phase 1: Data Preprocessing & "Scrubbing" (CPU & Pandas)

Scripts Involved: 01_pipeline.py
When the pipeline begins, the processing is entirely bound to the CPU and RAM, utilizing the pandas library (which runs highly optimized C-code under the hood via NumPy).

1. **Memory Ingestion (Merging):**
    - The processor reads 7 separate CSV files containing a total of ~102,000 rows. The CSV reader is instructed to use the engine="c" flag, forcing the CPU to parse the text bytes directly using C-bindings for speed.
    - These 7 datasets are concatenated into a massive matrix residing in system RAM (merged_raw.csv).
2. **Cross-Class Deduplication (Hash Tables):**
    - The processor builds a hash table of all author IDs. It counts the number of distinct labels (subreddit) associated with each author.
    - Any author whose hash count is > 1 (meaning they posted in both r/depression and r/ptsd, for example) is purged from the matrix. This requires scanning the entire index in memory and dropping ~3,359 users to prevent data leakage (where a user's unique writing style appears in both train and test sets).
3. **Downsampling & Splitting (Pseudorandom Number Generation):**
    - The control subreddits (Jokes, personalfinance, teaching) dominate the dataset. The CPU uses a pseudorandom number generator (seeded with random_state=42 to ensure deterministic execution) to randomly drop control rows until the control class drops to ~25,000 rows.
    - The data is then split by unique author into an 80% Training Matrix and a 20% Testing Matrix.
4. **The Scrubbing (Regular Expressions Engine):**
    - This is highly CPU-intensive. The CPU iterates over all ~70,000 remaining text strings.
    - For every single post, a Regular Expression (Regex) state machine is compiled. The CPU searches for exact byte-matches of the artifact terms (e.g., ptsd, diagnosed, r/anxiety).
    - When a match is found, the CPU allocates a new string in memory with the matched bytes replaced by empty space. This creates two distinct string columns in RAM: text_unscrubbed (raw) and text_scrubbed (modified).

Phase 2: Feature Engineering & Baseline Training (CPU & Scikit-Learn/XGBoost)

Scripts Involved: 02_train.py

1. **TF-IDF Vectorization:**
    - The CPU iterates over the raw text and tokenizes it by whitespace and punctuation.
    - It builds a sparse matrix (to save RAM) where rows are documents and columns are the 10,000 most frequent words in the entire corpus.
    - For every word in every document, it calculates the **Term Frequency (TF)** (how often the word appears in the document) multiplied by the **Inverse Document**

Frequency (IDF) (a penalty applied to words that appear in almost all documents, like "the" or "and").

2. **XGBoost (Gradient Boosting Trees):**
    - XGBoost utilizes a highly optimized C++ backend. The CPU builds 150 sequential decision trees (n_estimators=150).
    - Tree 1 is built to predict the class. It calculates the error (residuals).
    - Tree 2 is built specifically to predict and correct the *errors* made by Tree 1.
    - This gradient boosting process loops 150 times, resulting in a robust baseline model.

Phase 3: Deep Learning & GPU Acceleration (PyTorch & Transformers)

Scripts Involved: 08b_roberta.py, 08c_mentalbert.py, 08a_bilstm.py
This is where the massive Kaggle GPUs (Nvidia T4 x2) take over. A standard CPU would take weeks to process this; the GPUs do it in hours.

1. **Subword Tokenization (CPU):**
    - Deep learning models don't read words; they read vectors. The HuggingFace AutoTokenizer takes the strings and breaks them into "WordPieces". For example, "unhappiness" might become ["un", "##happi", "##ness"].
    - Each piece is mapped to an integer ID from the model's vocabulary (e.g., ID 4598).
    - The text is padded with zeros or truncated so every single post is exactly 128 tokens long (max_length=128). This creates a perfectly rectangular tensor (matrix) of integers, e.g., (56465 rows, 128 columns).
2. **Memory Transfer (PCIe Bus):**
    - These integer tensors are transferred from the system RAM across the motherboard's PCIe bus directly into the VRAM (Video RAM) of the Nvidia T4 GPUs.
3. **The Forward Pass (GPU Tensor Cores):**
    - Inside the GPU, the integer IDs are mapped to 768-dimensional floating-point vectors (embeddings).
    - In **RoBERTa and MentalBERT**, these embeddings pass through 12 Transformer layers. In each layer, "Self-Attention" matrices are multiplied. The GPU calculates how much every token in the 128-word sequence should "pay attention" to every other token. This requires massive parallel matrix multiplication (using the GPU's Tensor Cores).
    - The final output is pooled into a single 768-dimensional vector representing the entire post, which is passed through a final linear layer to output 5 probabilities (one for each class).
4. **The Backward Pass (Gradient Descent):**
    - The GPU calculates the Loss (Cross-Entropy Loss)—how far off the predictions were from the actual labels.

- Using Calculus (Backpropagation), the GPU calculates the derivative (gradient) of the loss with respect to every single one of the model's 110+ million parameters.
- The AdamW optimizer steps the parameters slightly in the direction that minimizes the loss (controlled by the learning_rate=2e-5).
- This entire Forward/Backward process happens repeatedly in batches of 16 posts at a time, looping over the entire 40,000+ row dataset 3 times (EPOCHS=3).

Phase 4: Explainable AI (SHAP Extraction)

Scripts Involved: 09a_roberta_shap.py, 09b_mentalbert_shap.py
This is the most computationally expensive phase of the project, taking the models from "black boxes" to interpretable systems.

1. **The Masking Process:**
    - SHAP (SHapley Additive exPlanations) is based on Game Theory. It attempts to determine the exact numerical contribution of a single word to the final prediction.
    - To do this for a specific post, the SHAP algorithm dynamically creates hundreds of copies of the text. In each copy, it "masks" (hides) different combinations of words.
    - For example, it passes "I feel terrible today" to the GPU. Then it passes "[MASK] feel terrible today". Then "I [MASK] terrible today".
2. **Attribution Calculation:**
    - By observing how the model's output probability changes when specific words are hidden, SHAP calculates the marginal contribution of that word.
    - The GPU rapidly evaluates thousands of these masked sentences.
    - The result is a numeric weight (SHAP value) for every single subword token in the dataset. A positive SHAP value for "panic" toward the Anxiety class means the word "panic" actively pushed the model to predict Anxiety.
3. **Aggregation:**
    - Because RoBERTa splits words into subwords (e.g., "pan" + "##ic"), the script aggregates the SHAP values back together to map to the original whole word.
    - The CPU calculates the average absolute SHAP value for every word across the entire dataset to rank the "Top Features."

Phase 5: Clinical Alignment (The Novel SAS Metric)

Scripts Involved: 05_alignment.py
With the models trained and the SHAP values extracted, the final step is to calculate the Symptom Alignment Score (SAS).

1. **Lexicon Cross-Referencing:**
    - The CPU loads the exact item wording from validated psychiatric instruments (PHQ-9, GAD-7, PCL-5, SPIN).

- It uses the NLTK WordNetLemmatizer to reduce both the SHAP words and the Lexicon words to their base linguistic roots (e.g., "flashbacks" -> "flashback", "running" -> "run").

2. **SAS Computation:**
    - For a given class (e.g., PTSD), the processor looks at the Top 50 words by SHAP weight.
    - It loops through those 50 words. If the word exists in the PCL-5 lexicon (e.g., "trauma"), its SHAP weight is added to a running sum.
    - The final SAS is the sum of the SHAP weights of the *clinically aligned* words divided by the total sum of all Top 50 SHAP weights.
    - This is repeated for both the Unscrubbed and Scrubbed arms, proving that the SAS goes up when the artifact "cheat words" are removed.
3. **Final Output Rendering:**
    - The 06_figures.py and 10_figures_multimodel.py scripts use matplotlib and seaborn to render the numerical data into the high-resolution PNG charts (fig6 and fig7) that you downloaded, completing the pipeline.

Phase 6: Result Output (Classification Metrics)

While the GPU handles the heavy lifting of the model forward passes, the final evaluation of the model's performance (the exact accuracy and Macro-F1 scores) is handed back to the CPU using scikit-learn's classification_report function.
The CPU compares the array of 14,118 true test labels against the array of 14,118 predicted labels and computes the harmonic mean (F1 score) across all 5 classes. The final numerical outputs generated by the processor for all 4 models are:

1. **MentalBERT (Domain-Specific Contextual Model)**
    - **Unscrubbed:** 86.07% Accuracy (0.8455 Macro-F1)
    - **Scrubbed:** 81.70% Accuracy (0.7941 Macro-F1)
    - **Accuracy Drop:** -4.37%
2. **RoBERTa (General Contextual Model)**
    - **Unscrubbed:** 86.82% Accuracy (0.8548 Macro-F1)
    - **Scrubbed:** 82.85% Accuracy (0.8067 Macro-F1)
    - **Accuracy Drop:** -3.97%
3. **XGBoost (Traditional Machine Learning Baseline)**
    - **Unscrubbed:** 82.04% Accuracy (0.8032 Macro-F1)
    - **Scrubbed:** 76.55% Accuracy (0.7344 Macro-F1)
    - **Accuracy Drop:** -5.49%
4. **BiLSTM (Recurrent Sequence Model)**
    - **Unscrubbed:** 78.65% Accuracy (0.7511 Macro-F1)
    - **Scrubbed:** 73.89% Accuracy (0.7101 Macro-F1)
    - **Accuracy Drop:** -4.76%

These metrics are serialized into Python .pkl (Pickle) files, which the processor converts from binary byte-streams back into human-readable text when requested.